

实验三实验报告

1. 寻优的适应度函数

GA 算法二进制:

```
// 适应度函数 f
double f(struct individual popu) {
    double func1, func2, func;
    double y[n];
    double u, dcu, cz, b, wm;
    double cx, tt;

    for (int j = 1; j ≤ p; j++)
        y[j] = popu.x[j];

    u = pi * y[1] * 1000.0 / 60.0;
    cz = 200.0 / (y[1] * y[1] * (1.0 - y[2] * y[2]) * pi);
    dcu = 980.0 / (1.26 * u * 0.8);
    wm = exp(0.5 * log(cz * cz + (u - dcu / 2.0) * (u - dcu / 2.0)));
    b = 117600.0 / (y[4] * y[3] * 1.26 * 800.0 * wm);
    cx = 0.022 - 0.006 * pi * y[1] / y[3] / b + 0.018 * y[4] * y[4] + 0.02 * 2.0 * pi / (y[3] * (1.0 - y[2]));
    tt = pi * y[1] / y[3];

    r[1] = b / tt - 0.5;
    r[2] = 1.5 - b / tt;

    func1 = 1.0 - cx / y[4] * wm * wm / (u * cz) - 0.1 * y[2] * y[2] * (2.0 - y[2] * y[2]) * cz * cz / (u * dcu);
    func2 = 56.0 * log(u) + 10.0 * log(980.0 * y[1] * (1.0 - y[2]) * y[3] * exp(0.6 * log(b)) / 2.0) - 26.22;
    func = 100.0 * func1 - func2 + 200.0;
    return func;
}
```

```
// 另一个适应度函数 f2
double f2(struct individual popu) {
    double func1, func2, func;
    double y[n];
    double u, dcu, cz, b, wm;
    double cx;

    for (int j = 1; j ≤ p; j++)
        y[j] = popu.x[j];

    u = pi * y[1] * 1000.0 / 60.0;
    cz = 200.0 / (y[1] * y[1] * (1.0 - y[2] * y[2]) * pi);
    dcu = 980.0 / (1.26 * u * 0.8);
    wm = exp(0.5 * log(cz * cz + (u - dcu / 2.0) * (u - dcu / 2.0)));
    b = 117600.0 / (y[4] * y[3] * 1.26 * 800.0 * wm);
    cx = 0.022 - 0.006 * pi * y[1] / y[3] / b + 0.018 * y[4] * y[4] + 0.02 * 2.0 * pi / (y[3] * (1.0 - y[2]));

    func1 = 1.0 - cx / y[4] * wm * wm / (u * cz) - 0.1 * y[2] * y[2] * (2.0 - y[2] * y[2]) * cz * cz / (u * dcu);
    func2 = 56.0 * log(u) + 10.0 * log(980.0 * y[1] * (1.0 - y[2]) * y[3] * exp(0.6 * log(b)) / 2.0);
    func = 100.0 * func1 - func2 + 500.0;
    return func;
}
```

两个适配工程场景的符合适应度函数 f 和 f2, f2 为 f 的变体, 用于结果验证。

GA 算法检验:

```

// 定义适应度函数
double f(struct individual popu) {
    double func;
    // 检验函数1, 求全局极小值
    // func=100*(popu.x[0]*popu.x[0]-popu.x[1])*(popu.x[0]*popu.x[0]-popu.x[1])
    //      +(1-popu.x[0])*(1-popu.x[0]);
    // return(-func);
    // 检验函数2, 求全局极小值
    // func=(1+(popu.x[0]+popu.x[1]+1)*(popu.x[0]
    //      +popu.x[1]+1)*(19-14*popu.x[0]+3*popu.x[0]*popu.x[0]-14*popu.x[1]
    //      +6*popu.x[0]*popu.x[1]+3*popu.x[1]*popu.x[1]))*(30+(2*popu.x[0]-3*popu.x[1])
    //      *(2*popu.x[0]-3*popu.x[1])*(18-32*popu.x[0]+12*popu.x[0]*popu.x[0]+48*popu.x[1]
    //      -36*popu.x[0]*popu.x[1]+27*popu.x[1]*popu.x[1]));
    // return(-func);
    // 检验函数3, 求全局极大值
    func = 0.5 - (sin(sqrt(popu.x[0] * popu.x[0] + popu.x[1] * popu.x[1])) *
        sin(sqrt(popu.x[0] * popu.x[0] + popu.x[1] * popu.x[1])) - 0.5) /
        (1.0 + 0.001 * (popu.x[0] * popu.x[0] + popu.x[1] * popu.x[1]) *
        (1.0 + 0.001 * (popu.x[0] * popu.x[0] + popu.x[1] * popu.x[1])));
    return func;
}

```

三个适应度函数，检验函数 1，Rosenbrock 函数，求全局极小值，测试 GA 的全局寻优能力；检验函数 2，Goldstein-Price 函数，求全局极小值，测试 GA 处理强非线性、多峰函数的能力；检验函数 3，Schaffer N.2 函数，求全局极大值，测试 GA 跳出局部最优、避免早熟收敛的能力。

GA 算法实数：

```

// 适应度函数 (Six-Hump Camel Back 函数的变形, 求最大值)
double f(struct individual popu) {
    double func;
    double y[n];

    for (int j = 1; j ≤ p; j++) {
        int k = (j - 1) * 2 + 1;
        popu.x[j] = popu.chrom[k] + popu.chrom[k + 1]; // 实数编码 + 简单相加
        y[j] = popu.x[j];
    }

    // Six-Hump Camel Back 函数 (原为求最小值, 这里取 -func + 10000 转为求最大值)
    func = (4 - 2.1 * y[1] * y[1] + y[1] * y[1] * y[1] * y[1] / 3.0) * y[1] * y[1];
    func = func + y[1] * y[2] + (4 * y[2] * y[2] - 4) * y[2] * y[2];
    func = -func + 10000.0; // 转为最大化问题
    return func;
}

```

适应度函数给予 Six-Hump Camel Back 经典测试函数改造，核心目标求该函数的最小值，做了逆变换后为最大化适应度。

2. 区别

对比维度	GA 算法二进制	GA 算法检验	GA 算法实数
编码方式	二进制编码： chro	二进制编码：chrom 存储 0/1 整数	实数编码：chrom 存储 double 型实数

	m 存储 0/1 整数		
变量映射逻辑	x[j]由 10 位二进制 chrom 解码为实数	x[j]由 10 位二进制 chrom 解码为实数	$x[j]=chrom[(j-1)*2+1]+chrom[(j-1)*2+2]$
适应度函数	工程类复合函数	Rosenbrock/Goldstein-Price/Schaffer N.2	Six-Hump Camel Back
交叉逻辑	二进制片段交换	仅对非精英个体操作	实数数组片段交换
变异逻辑	位翻转	位翻转	随机设 0/1
精英策略	保留最优个体，迭代回填	保留 8%精英个体，仅优化非精英	保留单个最优个体，迭代回填
终止条件	固定迭代 25 次	最优-平均适应度<0.01 或迭代 200 次	最优-平均适应度<0.1 或迭代 100 次
数组索引	从 1 开始	从 0 开始	从 1 开始

3. 寻优步骤

GA 算法二进制：

- (1) 初始化：设置种群大小、二进制位长、交叉变异概率、固定迭代 25 次。
- (2) 种群初始化：输入变量数和范围，随机生成 $x[j]$ ，计算适应度 f 。
- (3) 适应度平移：减去最小适应度，避免负适应度影响轮盘赌选择。
- (4) 编码：将 $x[j]$ 的实数值转为 10 位二进制，存入 $chrom$ 数组。
- (5) 排序：按适应度降序排列种群。
- (6) 局部优化：对最优个体调整 $x[j]$ 步长 0.1，迭代提升适应度。
- (7) 精英保留：保存最优个体到临时变量。
- (8) 迭代寻优：
 - 选择：轮盘赌选择个体，复制到旧种群；
 - 交叉：随机选两个个体，交换二进制位片段；
 - 变异：随机选位，翻转二进制位片段；
 - 解码：将二进制 $chrom$ 转回 $x[j]$ ，重新计算适应度；
 - 再次平移适应度，循环迭代。
- (9) 终止后：输出最优变量、 f_2 适应度，并用预设测试点验证结果。

GA 算法实数：

- (1) 初始化：设置种群大小、交叉变异概率、最大迭代等参数。
- (2) 种群初始化：输入变量数和范围，为每个个体生成实数 $chrom$ ，计算 $x[j]$ 和适应度。
- (3) 排序：按适应度降序排列种群。
- (4) 局部优化：对最优个体调整 $x[j]$ 步长 0.01，迭代提升适应度。
- (5) 精英保留：保存最优个体到临时变量。
- (6) 迭代寻优：
 - 选择：轮盘赌选择个体，复制到新种群；
 - 交叉：随机选两个个体，交换 $chrom$ 数组片段；
 - 变异：随机选位，将 $chrom$ 值设为 0/1；
 - 重新排序，回填精英个体，更新适应度。
- (7) 终止后：再次局部优化最优个体，输出变量和原函数最小值。